

SCOUT: Semantic Checkpointed Exploration for Unknown Indoor Environments with LLM-Based Guidance

Aman Sa, Dhyey Shah, Tao Wu, and Zijin Zhou
University of Pennsylvania

github.com/wu52F/F1-Project-SCOUT

Abstract

SCOUT is a semantic checkpointed exploration system for unknown indoor navigation with an Ackermann-steered robot. It combines continuous SLAM, reactive local planning, frontier/checkpoint pruning, camera-based POI memory, and constrained LLM/VLM guidance at junctions. The model only selects valid branch or checkpoint tokens, while the reactive planner preserves motion safety; this lets semantic guidance reduce loops and choose meaningful frontiers without issuing low-level commands.

1 Introduction

Unknown indoor exploration requires mapping, collision avoidance, goal selection, and recovery from dead ends. Ackermann vehicles add curvature limits and cannot rotate in place, so the planner must choose feasible forward trajectories rather than holonomic point motions.

SCOUT/MACRO separates the problem into small layers. SLAM provides the map and pose, **macro node** selects safe local motion, **exploration node** manages checkpoints and frontiers, and the VLM/keyboard interface only selects from allowed branch tokens. Camera observations are stored as POIs so the final map can explain what the robot saw at different visited places.

2 Problem Definition

The robot state is

$$\mathbf{x}_t = [x_t, y_t, \theta_t, v_t]^\top, \quad (1)$$

with control $\mathbf{u}_t = [a_t, \delta_t]^\top$. The goal is to build an occupancy map M_t , visit reachable unknown frontiers, avoid collisions, and recover to useful checkpoints when local progress fails.

The main constraint is command authority: semantic modules may suggest intent, but only the local planner publishes `/drive`. Branch decisions are restricted to tokens such as `left`, `straight`, `right`, `back`, and `stop`. This keeps VLM or keyboard guidance useful without allowing arbitrary unsafe motion.

2.1 Problem 1: Semantically Guided Hallway Choice

Problem 1 asks the car to use both the partial SLAM map and the forward camera view to decide which hallway branch is most likely to reach Skirkanich Hall and the staircase. As shown in Fig. 1, the robot begins near a junction where several geometric openings look plausible from LiDAR alone. A reactive follow-the-gap controller can choose the locally open left gap and then keep cycling through the same hallway geometry. Frontier navigation improves this by treating the map as a graph of unknown boundaries, but it can still explore extra junction branches before pruning the correct route. SCOUT uses the image prompt and map labels as semantic cues, so the branch decision is pruned directly toward the hallway associated with Skirkanich Hall and the stairs while the local planner still enforces collision safety.

2.2 Problem 2: Object-Conditioned Place Search

Problem 2 asks the car to find a remembered object rather than only a named destination. As shown

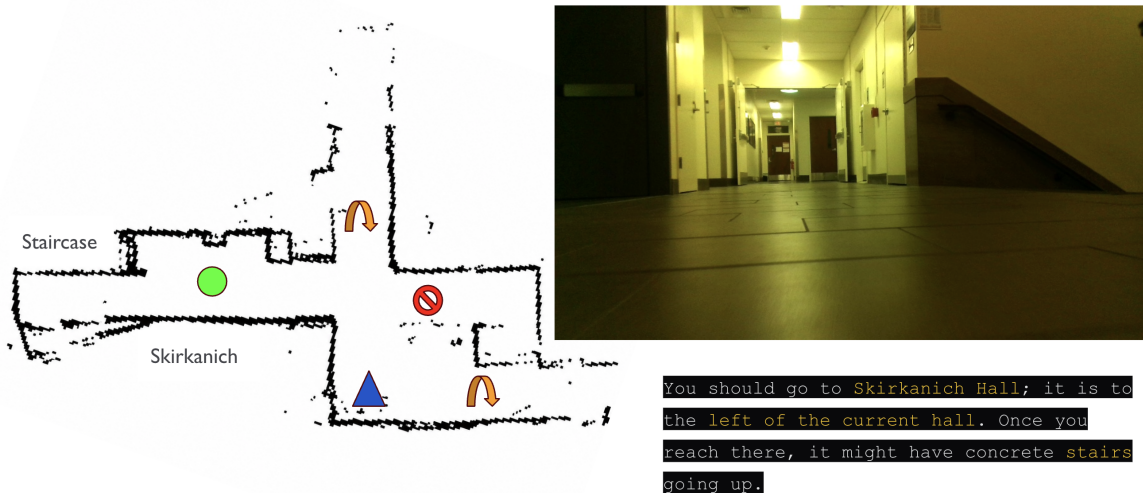


Figure 1: Problem 1 setup. The map view marks the robot, candidate turns, and the Skirkanich/staircase target region; the camera view and prompt provide semantic evidence that the useful branch is toward Skirkanich Hall rather than a locally attractive loop.

in Fig. 2, the prompt describes a yellow Amazon box that was previously observed after the green and black gas cylinders. This requires SCOUT to use POI memory as a searchable index: the gas cylinders provide an intermediate semantic landmark, while the yellow box is the final target. The robot must therefore choose branches that move through the remembered landmark sequence, prune branches that do not match the hint, and continue frontier exploration only where the map still offers a plausible route to the object.

2.3 Problem 3: Landmark-Guided Route Selection

Problem 3 asks the car to reach the elevator while avoiding a misleading or unavailable route. As shown in Fig. 3, the instruction says to take the longer hallway and expect the elevator on the left. The setup also includes an XLab-side corridor that is manually closed because of LiDAR limitations, so the robot must not treat every geometric opening as a valid path. SCOUT uses the map, branch status, and semantic prompt together: closed or unreliable branches are pruned, the longer hallway remains a valid candidate, and the camera/POI stream can confirm the elevator-side landmark before mission completion.

3 Methodology and Approach

The overall flow is shown in Fig. 4. Sensor streams feed mapping, local planning, semantic POI extraction, and exploration state. Exploration provides branch or frontier intent; the local planner remains the final safety gate. The SLAM Toolbox plus frontier-exploration reference implementation informed our map/frontier interface: the exploration layer subscribes to the occupancy grid, extracts frontiers at free/unknown boundaries, and repeatedly sends feasible exploration goals while SLAM updates the map [7].

A trial proceeds in repeated cycles: update SLAM, score local trajectories, refresh checkpoints/frontiers, select a branch or frontier, and trigger recovery if the robot becomes blocked. POIs are attached to map poses at selected visits so a later report can connect geometry with camera evidence.

4 System Design and Modeling

4.1 Reactive Exploration Loop

SCOUT is designed around a simple loop: move safely, update the map, identify what remains unexplored, and choose the next useful direction. SLAM



You should try to find the yellow
amazon box, I remember seeing it
after the green and black gas
cylinders.

Figure 2: Problem 2 setup. The map marks candidate turns, the robot pose, the gas-cylinder landmark, and the yellow Amazon box target; the camera view and prompt require the robot to connect current visual evidence with remembered POIs.

updates the robot pose and occupancy grid at every step, so each decision is made using the latest estimate of free space, walls, and unknown boundaries. The local planner then converts LiDAR into an inflated costmap,

$$C(x, y) = C_{\max} \exp \left(-\frac{d(x, y)^2}{2\sigma^2} \right), \quad (2)$$

and scores safe candidate trajectories by

$$J_i = w_s J_s + \alpha w_p J_p + w_c J_c + w_l J_l + w_b J_b. \quad (3)$$

The bias term J_b softly favors an exploration direction, but obstacle rejection remains hard. This means SCOUT can reactively drive through newly observed space while still allowing the higher-level exploration layer to express intent.

For the Ackermann platform, motion is constrained by the bicycle model

$$x_{t+1} = x_t + \Delta t v_t \cos \theta_t, \quad y_{t+1} = y_t + \Delta t v_t \sin \theta_t, \quad (4)$$

$$\theta_{t+1} = \theta_t + \Delta t \frac{v_t}{L} \tan \delta_t. \quad (5)$$

Thus every chosen path must remain feasible for forward motion and bounded steering, rather than assuming the robot can rotate in place.

4.2 Free-Unknown Boundaries and Pruning

The exploration layer interprets the SLAM map by looking for frontiers: free cells that touch unknown cells. These free-unknown boundaries are natural exploration goals because crossing toward them is likely to reveal new rooms, corridors, or junctions. Checkpoints summarize places the car can return to, while frontier clusters summarize areas that still deserve exploration. A candidate frontier can be ranked by

$$U(q) = \lambda_u I(q) - \lambda_d D(q) - \lambda_r R(q), \quad (6)$$

where information gain I is balanced against travel effort D and retry penalty R .

When only one useful frontier or branch remains, SCOUT does not need a semantic decision: it pursues that area directly. As the car reaches a frontier, exhausts a branch, or detects that a path is blocked, the exploration graph is pruned so the same failed region is not repeatedly selected. This pruning keeps the car from looping and turns the map into short-term memory: explored branches are marked as consumed, failed branches are penalized, and remaining frontiers become the next candidates.

4.3 Branches and Junctions

A junction is treated as a checkpoint where the local map contains more than one reachable branch.

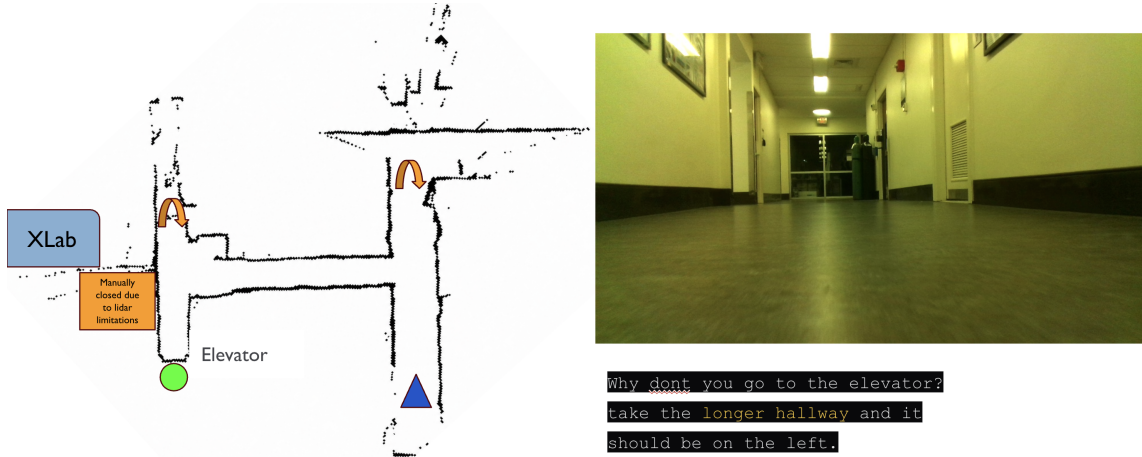


Figure 3: Problem 3 setup. The map highlights the elevator target, the longer hallway route, and an XLab-side branch closed because of LiDAR limitations; the prompt guides branch selection toward the elevator on the left.

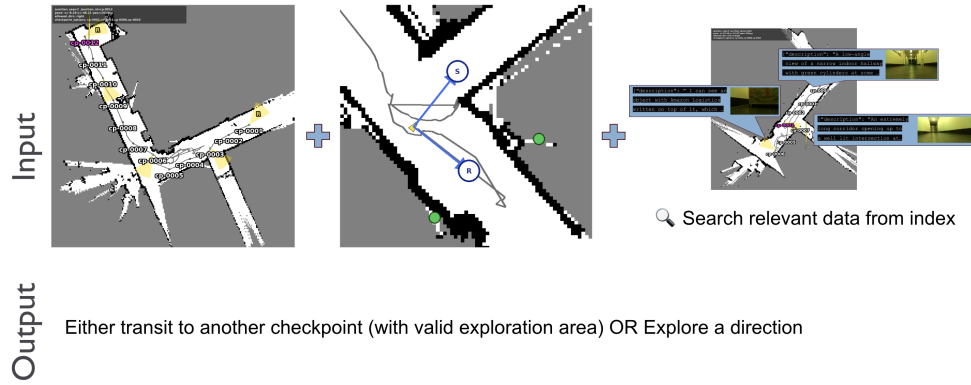


Figure 4: Full SCOUT/MACRO flow diagram. The naming and arrows show the intended execution order: sensing, mapping, local planning, exploration decisions, branch selection, and POI memory.

Each branch is represented by its direction, nearby frontier cells, checkpoint anchor, and status. The branch status records whether it is open, already explored, blocked, or waiting for a higher-level decision. This representation lets SCOUT separate geometry from intent: the map determines which branches are feasible, while the exploration layer decides which one should be tried next.

When a branch has a useful frontier, the robot can bias the local planner toward that opening. When a branch has no remaining frontier, repeatedly fails, or has been marked blocked, it is pruned from the active candidate set. This prevents the car from revisiting the same dead end and keeps later decisions focused on branches that can still reveal new space.

4.4 A* Pursuit for Branch Pruning

A* pursuit is used when pruning requires the robot to move through known checkpoints instead of simply choosing the closest visible gap. If a selected branch needs to be verified, exhausted, or connected to a safer frontier, A* searches the checkpoint graph with

$$f(n) = g(n) + h(n), \quad h(n) = \|p_n - p_{goal}\|_2. \quad (7)$$

The resulting path is not treated as direct actuation. It is a sequence of checkpoint targets that the local planner follows while still rejecting unsafe LiDAR observations. If the car reaches the branch and finds no useful frontier, the branch is pruned; if it reaches a new frontier, the branch remains active until that

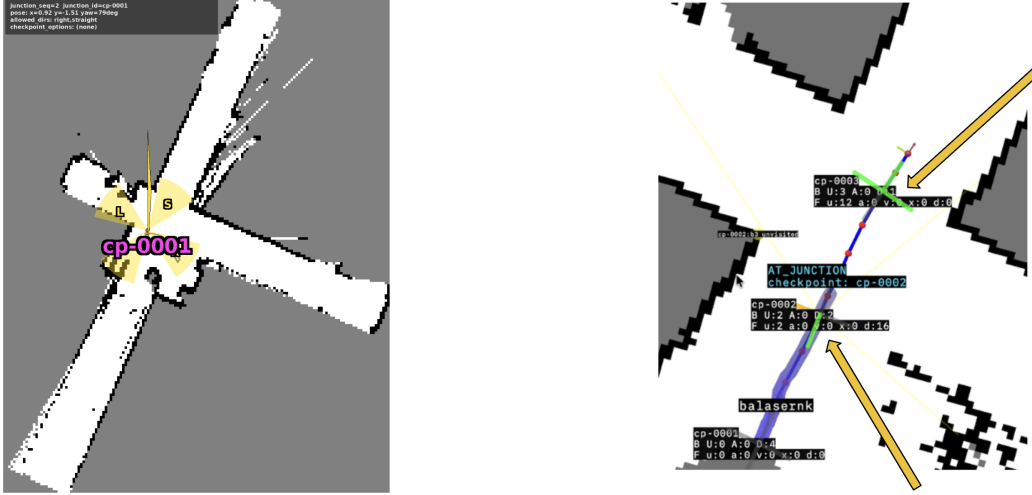


Figure 5: Checkpoint and frontier views used by the exploration layer. Left: junction/frontier geometry identifies allowed branch choices and nearby unknown boundaries. Right: visited checkpoints become reusable anchors for pruning, recovery, and VLM-guided branch decisions.

frontier is consumed.

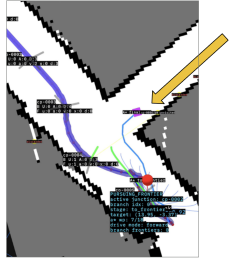


Figure 6: A* pursuit transit from the current checkpoint to a safer goal.

4.5 Recovery

Recovery handles cases where the reactive planner cannot make progress, the selected branch becomes blocked, or the checkpoint path cannot be followed safely. The first response is conservative: stop, rewind to a recent valid checkpoint if possible, and refresh the local map. If the branch still appears blocked after the persistence window, the system penalizes or prunes that branch and chooses another candidate from the checkpoint graph.

This recovery mode is intentionally separate from A* pursuit. A* gives a planned route through known space, while recovery decides what to do when that route or the local planner fails. Keeping the two roles separate makes failures easier to explain in logs: a branch can be marked failed be-

cause transit timed out, because all local trajectories were rejected, or because no frontier remained after arrival.

4.6 POI Creation as a Place Index

Camera POIs are stored as tuples $\rho_i = (p_i, \theta_i, t_i, \eta_i, z_i)$, where p_i is map pose, η_i is the image or embedding, and z_i is a semantic label such as door, sign, corridor, room, or obstacle. Together these records form an index of visited places. Each POI links what the robot saw to where it was in the checkpoint graph, so the system can later search for semantic hints such as a hallway sign, staircase, doorway, or remembered object.

The POI index supports decision making without controlling the car directly. A matched place can raise the score of a nearby branch, explain why a previous turn was useful, or provide evidence that a mission hint has been satisfied. The local planner remains responsible for motion, while POIs provide searchable context for choosing among map options.

4.7 VLM Planning and Pruning

When multiple branches exist at a junction, VLM planning acts as the high-level guide. The system sends a compact context containing the mission hint, available branch tokens, checkpoint IDs, recent POIs, and the annotated map. The model must re-

turn one allowed option, such as **left**, **straight**, **right**, or a checkpoint identifier; the exploration node then converts that option into a branch bias or frontier target.

This makes the VLM a planning and pruning assistant rather than a controller. It can use POI evidence and mission language to choose the branch most likely to reach a useful place, but it cannot invent unsafe actions. If the chosen branch is explored and no longer contains useful frontier cells, the branch is pruned and the next decision is made from the remaining options.

4.8 Mission Completion

Mission completion is checked continuously from both geometric progress and semantic evidence. The POI index compares the current camera observations against earlier POIs and mission hints, so repeated evidence from the same hallway, sign, room, or object can indicate that the requested place has been found. Completion can also be triggered when all reachable frontiers have been consumed or every remaining branch has been pruned.

Once the mission is complete, the exploration layer stops requesting new branches and can publish a return-home intent. The car then follows known checkpoints back through the map using the same safety gate as normal exploration. This keeps completion, return-home behavior, and collision avoidance in the same checkpoint-frontier framework.

5 Implementation

The implementation uses explicit ROS 2 interfaces rather than hidden coupling. Inputs are `/scan`, `/odom`, `/map`, TF, and camera frames. Outputs are `/drive`, `/macro/open_ratio`, `/macro/all_blocked`, `/exploration/branch_bias`, `/vlm/decision`, and POI records. Figure 8 combines these interfaces into the end-to-end runtime workflow.

The exploration state machine has four practical modes: explore, plan, recover, and complete. In explore mode, the robot follows frontiers and records checkpoints. In plan mode, a junction option set is sent to the VLM or keyboard. In recover mode, the robot rewinds or runs A* over the checkpoint graph. In complete mode, no useful frontier remains or return-home has been requested.

Three invariants guide the implementation. First, only the local planner publishes actuation. Second, every branch target passes through local trajectory rejection. Third, recovery actions must stop or return to known checkpoints if transit planning fails. These invariants make logs easier to debug because safety, intent, and semantics remain separated.

6 Parameters and Configuration

The most important parameters are costmap resolution, inflation width, collision threshold, trajectory horizon, lattice sample count, speed limit, steering limit, checkpoint spacing, frontier size threshold,

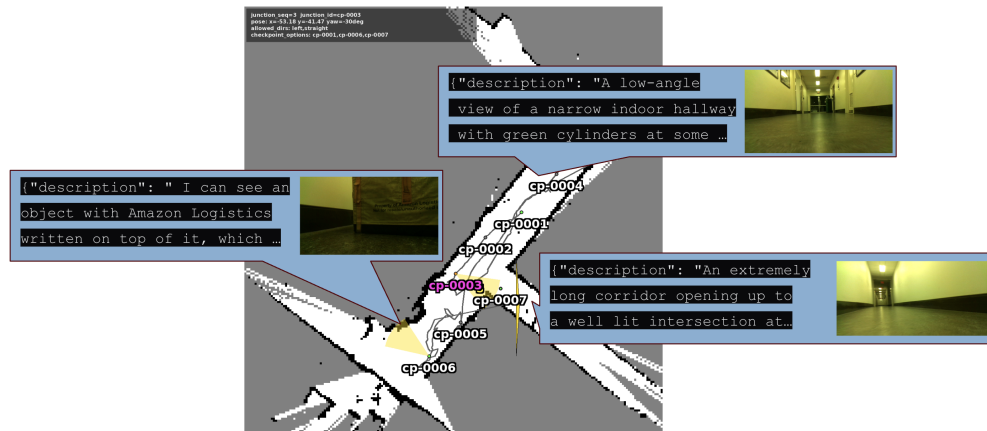


Figure 7: POI map flow: camera images collected at different visited places are linked to map pose and summarized into semantic landmarks.

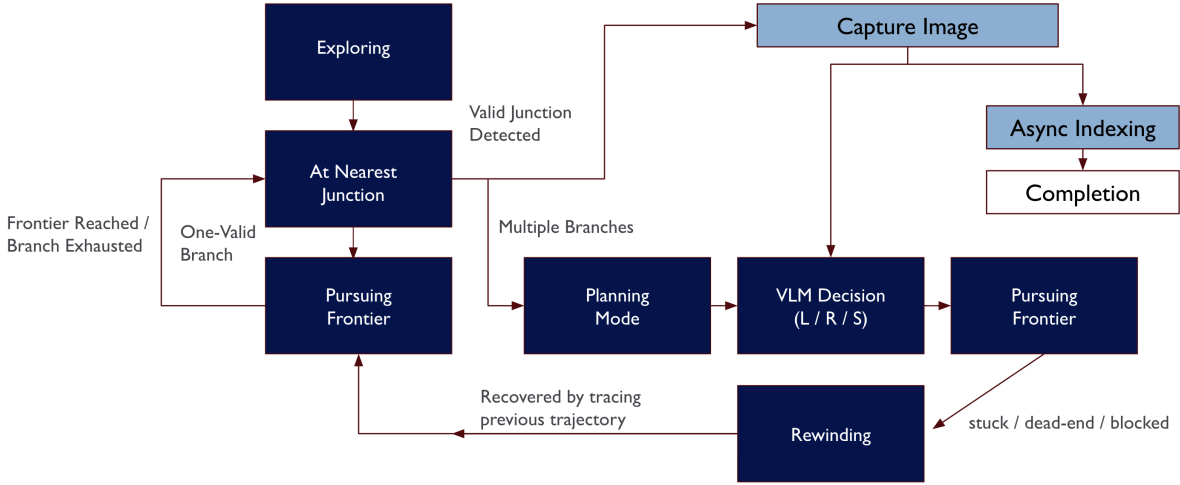


Figure 8: Implementation state machine. The exploration layer alternates between frontier exploration, junction planning, reactive safety gating, and checkpoint/A* recovery; camera POI memory and VLM or keyboard branch selection provide context without bypassing the final local safety gate before `/drive`.

branch angle threshold, blocked persistence window, VLM timeout, and POI image cadence.

Table 1: Compact configuration checklist.

Group	Report
Local safety	Costmap size, inflation, collision threshold
Trajectory	Horizon, samples, weights, speed and steering limits
Exploration	Checkpoint spacing, frontier threshold, branch angle
Recovery	Blocked window, rewind length, A* transit timeout
VLM/POI	Allowed tokens, timeout, image cadence, fallback mode

Tuning should start with local safety, then trajectory smoothness, then exploration thresholds. Otherwise a global exploration failure may actually be caused by an over-conservative or under-inflated local planner.

7 Evaluation and Results

We evaluated SCOUT on the three problem setups introduced in Section 2. Each run tested whether the robot could combine local geometry with semantic instructions, POI evidence, and branch pruning rather than simply selecting the nearest open gap. The successful behavior across all three setups is summarized in Table 2.

These experiments demonstrate that SCOUT can

preserve semantic intent across multiple junctions, reason over delayed landmark-conditioned actions, and use visual semantic cues for mission completion instead of relying only on local frontier selection. The checkpoint-frontier representation allowed the system to maintain branch consistency while remaining compatible with the reactive safety and exploration pipeline.

The failure checklist should still identify SLAM drift, unreachable frontier, local oscillation, blocked-path persistence, invalid VLM token, late decision, and POI mismatch. Reporting the observable evidence for each failure is more useful than a single navigation-failed label.

8 Assumptions and Limitations

The stack assumes reliable LiDAR, odometry, camera frames, map-frame transforms, and mostly planar indoor structure. It remains sensitive to SLAM drift, poor inflation tuning, wide-intersection ambiguity, sparse frontiers, and remote VLM latency. POI labels are descriptive annotations; they should not directly steer the vehicle without geometric validation.

The current camera is forward facing, so POI creation is biased toward whatever appears in front of the car at the moment a checkpoint is recorded.

Table 2: Evaluation across the three semantic navigation problem setups.

Setup	Instruction Challenge	Observed SCOUT Behavior	Outcome
Problem 1	The robot is first offered front and right choices, but the correct route requires going straight so it can take the left branch afterward.	SCOUT did not greedily turn at the first visible choice. It continued straight, preserved the next junction as the useful branch, and then selected the left turn toward the Skirkanich/staircase route.	Successful semantic branch sequencing.
Problem 2	The prompt says to take a turn after the green cylinders, so seeing the cylinders is not itself enough to turn immediately.	SCOUT recognized that the green cylinders had just appeared and kept going straight until the landmark was passed. It then took the right turn associated with the remembered yellow Amazon box route.	Successful POI-conditioned delayed turn.
Problem 3	The instruction is more semantic: “take the longer hallway” and finish when an elevator door is observed.	SCOUT favored the longer hallway over shorter misleading branches and used the elevator-door observation as the completion cue rather than only relying on frontier exhaustion.	Successful landmark-guided mission completion.

This can miss signs, doors, objects, or hallway landmarks that are beside or behind the vehicle. A stronger POI memory system would use a 360-degree camera or multiple synchronized views so each visited place is indexed with more complete visual context before later VLM search and mission-completion checks.

9 Conclusion and Future Work

SCOUT/MACRO combines SLAM, reactive Ackermann control, checkpoint-frontier reasoning, A* recovery, camera-based POI memory, and optional VLM branch selection. The diagram-driven structure makes the stack easier to explain: each flow shows one responsibility and the data contract between modules.

Future work should evaluate larger maps, stronger SLAM-drift detection, learned branch utility, uncertainty-aware frontier scoring, and POI-assisted return-to-object tasks. The next experiments should compare VLM, heuristic, and human branch choices under identical allowed-token constraints.

Acknowledgment

The authors thank the project contributors and reviewers who supported development, testing, and documentation of the SCOUT/MACRO exploration stack. They also thank everyone who was asked to step aside as the car crashed into the hundred different things while bug fixing.

References

- [1] G. Grisetti, R. Kummerle, C. Stachniss, and W. Burgard, “A tutorial on graph-based SLAM,” *IEEE Intelligent Transportation Systems Magazine*, vol. 2, no. 4, pp. 31–43, 2010.
- [2] B. Yamauchi, “A frontier-based approach for autonomous exploration,” in *IEEE CIRA*, 1997, pp. 146–151.
- [3] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [4] S. Thrun et al., “Stanley: The robot that won the DARPA Grand Challenge,” *Journal of Field Robotics*, vol. 23, no. 9, pp. 661–692, 2006.
- [5] S. Macenski and I. Jambrecic, “SLAM Toolbox: SLAM for the dynamic world,” *Journal of Open Source Software*, vol. 6, no. 61, 2021.
- [6] D. Fox, W. Burgard, and S. Thrun, “The dynamic window approach to collision avoidance,” *IEEE Robotics and Automation Magazine*, vol. 4, no. 1, pp. 23–33, 1997.
- [7] A. Arka, “Autonomous Explorer and Mapper ROS2 Nav2,” GitHub repository, 2025. [Online]. Available: <https://github.com/AniArka/Autonomous-Explorer-and-Mapper-ros2-nav2>. Accessed: May 13, 2026.